

## 1. A\* Using Misplaced Search

```
import heapq

GOAL_STATE = (1, 2, 3, 8, 0, 4, 7, 6, 5)

def misplaced_tiles(state):
    return sum(s != 0 and s != g for s, g in zip(state, GOAL_STATE))

def get_neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    neighbors = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        if 0 <= r + dr < 3 and 0 <= c + dc < 3:
            nz = (r + dr) * 3 + (c + dc)
            lst = list(state)
            lst[z], lst[nz] = lst[nz], lst[z]
            neighbors.append(tuple(lst))
    return neighbors

def count_inversions(state):
    s = [t for t in state if t]
    return sum(s[i] > s[j] for i in range(len(s)) for j in range(i + 1, len(s)))

def is_solvable(start_state):
    return count_inversions(start_state) % 2 == count_inversions(GOAL_STATE) % 2

def a_star_misplaced(start_state):
    pq = [(misplaced_tiles(start_state), 0, start_state, [start_state])]
    visited = {start_state: 0}
    while pq:
        f, g, cur, path = heapq.heappop(pq)
        if cur == GOAL_STATE:
            return path
        for nbr in get_neighbors(cur):
            new_g = g + 1
            if nbr not in visited or new_g < visited[nbr]:
```

```

        visited[nbr] = new_g
        heapq.heappush(pq, (new_g + misplaced_tiles(nbr), new_g,
nbr, path + [nbr]))
    return None

def print_board(state):
    for i in range(0, 9, 3):
        print(" ".join(str(v) if v else ' ' for v in state[i:i+3]))
    print("-" * 9)

if __name__ == "__main__":
    print("Enter the 8-puzzle board layout using numbers 0 to 8 (where 0
represents the blank space).")
    print("You can enter all 9 numbers separated by spaces.\n")

    while True:
        try:
            nums = [int(x) for x in input("Enter 9 numbers: ").split()]
            if len(nums) != 9:
                print("Error: You must enter exactly 9 numbers. Try
again.")
                continue
            if sorted(nums) != list(range(9)):
                print("Error: Numbers must include all integers from 0 to
8 without duplicates. Try again.")
                continue
            initial_state = tuple(nums)
            break
        except ValueError:
            print("Error: Invalid input. Please enter integers only.")

    print("\nInitial State Entered:")
    print_board(initial_state)
    print("Target Goal State:")
    print_board(GOAL_STATE)

    if not is_solvable(initial_state):
        print("This puzzle configuration is UNSOLVABLE (mismatched
inversion parity with the goal state).")
    else:

```

```

print("Solvability check passed! Solving using A* Search...\n")
solution = a_star_misplaced(initial_state)

if solution:
    print(f"Solution found in {len(solution) - 1} steps!\n")
    for step, state in enumerate(solution):
        print(f"Step {step}:")
        print_board(state)
    else:
        print("No solution found.")

```

Output:

```

Enter the 8-puzzle board layout using numbers 0 to 8 (where 0 represents the blank space).
1 6 4
7 5
-----
Target Goal State:
1 2 3
8 4
7 6 5
-----
Solvability check passed! Solving using A* Search...

Solution found in 5 steps!

Step 0:
2 8 3
1 6 4
7 5
-----
Step 1:
2 8 3
1 4
7 6 5
-----
Step 2:
2 3
1 8 4
7 6 5
-----
Step 3:
2 3
1 8 4
7 6 5
-----
Step 4:
1 2 3
8 4
7 6 5
-----
Step 5:
1 2 3
8 4
7 6 5
-----
PS C:\Users\student\Desktop\1WA24CS205>

```

## 2. A\* Using Manhattan Search

```
import heapq

GOAL_STATE = (1, 2, 3, 8, 0, 4, 7, 6, 5)
GOAL_POS = {t: divmod(i, 3) for i, t in enumerate(GOAL_STATE) if t != 0}

def manhattan_distance(state):
    return sum(abs(i // 3 - GOAL_POS[t][0]) + abs(i % 3 - GOAL_POS[t][1])
               for i, t in enumerate(state) if t != 0)

def get_neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    neighbors = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        if 0 <= r + dr < 3 and 0 <= c + dc < 3:
            nz = (r + dr) * 3 + (c + dc)
            lst = list(state)
            lst[z], lst[nz] = lst[nz], lst[z]
            neighbors.append(tuple(lst))
    return neighbors

def count_inversions(state):
    s = [t for t in state if t]
    return sum(s[i] > s[j] for i in range(len(s)) for j in range(i + 1,
len(s)))

def is_solvable(start_state):
    return count_inversions(start_state) % 2 ==
count_inversions(GOAL_STATE) % 2

def a_star_manhattan(start_state):
    pq = [(manhattan_distance(start_state), 0, start_state,
[start_state])]
    visited = {start_state: 0}
    while pq:
        f, g, cur, path = heapq.heappop(pq)
        if cur == GOAL_STATE:
            return path
```

```

        for nbr in get_neighbors(cur):
            new_g = g + 1
            if nbr not in visited or new_g < visited[nbr]:
                visited[nbr] = new_g
                heapq.heappush(pq, (new_g + manhattan_distance(nbr),
new_g, nbr, path + [nbr]))
            return None

def print_board(state):
    for i in range(0, 9, 3):
        print(" ".join(str(v) if v else ' ' for v in state[i:i+3]))
    print("-" * 9)

if __name__ == "__main__":
    print("Enter the 8-puzzle board layout using numbers 0 to 8 (where 0
represents the blank space).")
    print("You can enter all 9 numbers separated by spaces.\n")

    while True:
        try:
            nums = [int(x) for x in input("Enter 9 numbers: ").split()]
            if len(nums) != 9:
                print("Error: You must enter exactly 9 numbers. Try
again.")
                continue
            if sorted(nums) != list(range(9)):
                print("Error: Numbers must include all integers from 0 to
8 without duplicates. Try again.")
                continue
            initial_state = tuple(nums)
            break
        except ValueError:
            print("Error: Invalid input. Please enter integers only.")

    print("\nInitial State Entered:")
    print_board(initial_state)
    print("Target Goal State:")
    print_board(GOAL_STATE)

    if not is_solvable(initial_state):

```

```

        print("This puzzle configuration is UNSOLVABLE (mismatched
inversion parity with the goal state).")
    else:
        print("Solvability check passed! Solving using A* Search
(Manhattan Distance)...\n")
        solution = a_star_manhattan(initial_state)

        if solution:
            print(f"Solution found in {len(solution) - 1} steps!\n")
            for step, state in enumerate(solution):
                print(f"Step {step}:")
                print_board(state)
        else:
            print("No solution found.")

```

Output:

```

Enter the 8-puzzle board layout using numbers 0 to 8 (where 0 represents the blank space).
1 6 4
7 5
-----
Target Goal State:
1 2 3
8 4
7 6 5
-----
Solvability check passed! Solving using A* Search...

Solution found in 5 steps!

Step 0:
2 8 3
1 6 4
7 5
-----
Step 1:
2 8 3
1 4
7 6 5
-----
Step 2:
2 3
1 8 4
7 6 5
-----
Step 3:
2 3
1 8 4
7 6 5
-----
Step 4:
1 2 3
8 4
7 6 5
-----
Step 5:
1 2 3
8 4
7 6 5
-----
PS C:\Users\student\Desktop\1\A24CS205>

```